

->UART PROTOCOL-----

//BAUD_RATE

```
module baud_rate(input clk, rst, output reg tx_br, rx_br);
parameter clk_freq = 100;
parameter baud_rate=96;
parameter TX_MAX_COUNT= clk_freq/baud_rate;// 100mhz/9600
parameter RX_MAX_COUNT= clk_freq/baud_rate;
reg[31:0] tx_counter; // reg[$clog2(TX_MAX_COUNT) 1:0)
reg[31:0] rx_counter;
//TX BAUD-RATE
always@(posedge clk or posedge rst)begin
if(rst)begin
tx_br<=0;
tx_counter <=0;
end
else begin
if(tx_counter == (TX_MAX_COUNT)) begin
tx_br <=1;
tx_counter <=32'd0;
end
else begin
tx_counter <= tx_counter+1;
tx_br <=0;
end
end
end
always@(posedge clk or posedge rst)begin
if(rst)begin
rx_br<=0;
rx_counter<=32'd0;
```

```

end
else begin
if(rx_counter==(RX_MAX_COUNT))begin
rx_br <=1;
rx_counter <=32'd0;
end
else begin
rx_br <=0;
rx_counter <= rx_counter+1;
end
end
end
endmodule

```

//BAYD RATE TB:

```

module baud_tb( );
reg clk,rst;
wire tx_br;
wire rx_br;

baud_rate dut(clk,rst,tx_br,rx_br);

always #5 clk=~clk;
initial begin
clk=0;rst=1;
#10 rst=0;
#900 $finish;
end

```

endmodule

//UART_TX

```
module uart_tx(input clk,  
               input en,  
               input start,  
               input tx_br,  
               input [7:0]data,  
               output reg done,  
               output reg busy,  
               output reg tx_out );
```

```
parameter RESET =0;  
parameter IDLE=1;  
parameter START=2;  
parameter DATA=3;  
parameter PARITY=4;  
parameter STOP =5; //1012
```

```
// INTERNAL REG
```

```
reg [7:0]hs_reg; //hold&shift reg  
reg [2:0] state;  
reg parity ;  
reg [2:0]bit_index; //to count the data bits
```

```
always @ (posedge clk)begin  
if(!en)begin  
state<=RESET;  
end
```

```
else if(tx_br)begin
```

```
case(state)
```

```
RESET:begin
```

```
busy<=0;
```

```
done<=0;
```

```
bit_index<=0;
```

```
tx_out<=1; //always the transmission line of UART will be high
```

```
state<=IDLE;
```

```
end
```

```
IDLE:begin
```

```
if(start)begin
```

```
hs_reg <=data; //load the data to hold register
```

```
parity <=^data;
```

```
state <=START;
```

```
end
```

```
else
```

```
state<=IDLE;
```

```
end
```

```
START:begin
```

```
tx_out <=0;
```

```
busy <=1;
```

```
done=0;
```

```
state<=DATA;
```

```
end
```

```
DATA:begin
```

```
tx_out<=hs_reg[0];
```

```
hs_reg<={1'b0,hs_reg[7:1]}; //hs reg>>1;
```

```
bit_index <=bit_index+1;
if(bit_index==7)
state<=PARITY;
else
state<=DATA;
end
```

```
PARITY:begin
tx_out<=parity;
state<=STOP;
end
```

```
STOP:begin
tx_out <=1;
busy <=0;
done <=1;
if(start)
state<=IDLE;
else
state<=RESET;
end
endcase
end
end
endmodule
```

```
//UART TX TB:
```

```
module uart_tx_tb( );
reg clk,en,start;
```

```
reg[7:0]data;
```

```
wire done;
```

```
wire busy;
```

```
wire tx_out;
```

```
uart_tx dut(.clk(clk),
```

```
    .en(en),
```

```
    .start(start),
```

```
    .data(data),
```

```
    .done(done),
```

```
    .busy(busy),
```

```
    .tx_out(tx_out));
```

```
always #5 clk=~clk;
```

```
initial begin
```

```
    //$monitor("clk=%0b;en=%b;start=%b;data=%b;done=%b;busy=%b;tx  
out=%b",clk,en,start,data,done,bust,tx_out);
```

```
    clk=0;en=0;data=8'b00000000;start=0;
```

```
    #10 en=1;
```

```
    data=8'b11100011;
```

```
    start=1;
```

```
    wait(done);
```

```
    #10 start=0;
```

```
    #10 start=1;data=8'b11001100;
```

```
    wait(done);
```

```
    #10 start=0;
```

```
    #200 $finish;
```

```
end
```

```
endmodule
```

//UART_RX

```
module uart_rx(input clk,  
               input en,  
               input rx_br,  
               input rx_data, //1-bit  
               output reg[7:0] rx_out, //8-bit  
               output reg done,  
               output reg busy,  
               output reg error );
```

```
parameter RESET=0;
```

```
parameter START=1;
```

```
parameter DATA=2;
```

```
parameter PARITY=3;
```

```
parameter STOP=4;
```

```
parameter READY=5;
```

```
reg [2:0]state;
```

```
reg [7:0] sb_reg; //shift & buff reg
```

```
reg [2:0]bit_index;
```

```
always @(posedge clk)begin
```

```
if(!en)begin
```

```
state<=RESET;
```

```
end
```

```
else if(rx_br) begin
```

```
case(state)
```

```
RESET:begin
```

```
done <=0;
```

```
busy<=0;
```

```
error <=0;
sb_reg<=0;
//rx_data<=0;
bit_index<=0;
state <=START;
end
```

```
START:begin
if(!rx_data)begin //indicate the start bit
busy <=1;
state<=DATA;
end
else
state<=START;
end
```

```
DATA:begin
sb_reg <={rx_data,sb_reg[7:1]};
bit_index<=bit_index+1;
if (bit_index==7)
state <=PARITY;
else
state<=DATA;
end
```

```
PARITY:begin
error <=(^sb_reg != rx_data);
state <= STOP;
end
```

```
STOP:begin
```



```
if(rx_data)begin
done<=1;
rx_out <=sb_reg;
state<=READY;
end
else begin
error<=1;
state<=RESET;
end
end
```

```
READY:begin
busy <=0;
state<=RESET;
end
endcase
end
end
endmodule
```

//UART RX TB:

```
module uart_rx_tb( );
reg clk,en,rx_data;
wire [7:0]rx_out;
wire busy,done,error;

uart_rx dut(.clk(clk),
            .en(en),
            .rx_data(rx_data),
            .rx_out(rx_out),
```

```

        .busy(busy),
        .done(done),
        .error(error));

task send_data(input [7:0]data,input parity);
integer i;
begin
    rx_data = 0;
    #10;
    for(i=0;i<8;i=i+1)begin
        rx_data=data[i];
        #10;
    end
    rx_data=parity;
    #10;
    rx_data=1; //stopbit
    #10;
end
endtask

always #5 clk=~clk;

initial begin
    $monitor("clk:%0b;en:%b;rx-data:%b;sb-reg:%b;rx-out:%b;done:%b;busy:%b;error:%b",
        clk,en,rx_data,dut.sb_reg,rx_out,done,busy,error);
    clk=0;en=0;rx_data=1;
    #10 en=1;
    #10;
    send_data(8'b010101101,0);
    #10;
    $display("test completed");
    $display("received data:%b",rx_out);
    $stop;

```

```
end
```

```
endmodule
```

```
-----
```

```
//TOP MODULE
```

```
module uart_top(input clk,
```

```
    input rst,
```

```
    input en,
```

```
    input start,
```

```
    input [7:0]tx_data,
```

```
    output [7:0] rx_data,
```

```
    output tx_done,tx_busy,
```

```
    output rx_done,rx_busy,
```

```
    output rx_error );
```

```
wire tx_out; //connecting output from tx to input of rx
```

```
wire tx_br,rx_br;
```

```
baud_rate br(.clk(clk),.rst(rst),.tx_br(tx_br),.rx_br(rx_br));
```

```
uart_tx
```

```
tx(.clk(clk),.en(en),.start(start),.tx_br(tx_br),.data(tx_data),.tx_out(tx_out),.done(tx_done),.busy(tx_busy));
```

```
uart_rx
```

```
rx(.clk(clk),.en(en),.rx_br(rx_br),.rx_data(tx_out),.rx_out(rx_data),.done(rx_done),.busy(rx_busy),.error(rx_error));
```

```
endmodule
```

```
//TOP MODULE TB:
```

```
module top_tb();
```

```
reg clk;
```

```
reg rst;
```

```
reg en;
```

```
reg start;
```

```
reg [7:0]tx_data;
```

```
wire [7:0] rx_data;
```

```
wire tx_done,tx_busy;
```

```
wire rx_done,rx_busy;
```

```
wire rx_error ;
```

```
uart_top dut(.clk(clk),
```

```
    .rst(rst),
```

```
    .en(en),
```

```
    .start(start),
```

```
    .tx_data(tx_data),
```

```
    .rx_data(rx_data),
```

```
    .tx_done(tx_done),
```

```
    .rx_done(rx_done),
```

```
    .tx_busy(tx_busy),
```

```
    .rx_busy(rx_busy),
```

```
    .rx_error(rx_error));
```

```
always #5 clk=~clk;
```

```
initial begin
```

```
$monitor("rx_data:%b, rx_data:%b, tx_done:%b, rx_done:%b, tx_busy:%b, rx_busy:%b",
```

```
    tx_data, rx_data, tx_done, rx_done,tx_busy, rx_busy);
```

```
    clk=0;rst=1;en=0;start=0;tx_data=8'd0;
```

```
    #20 rst=0;en=1;
```

```
#20 t_bytes(8'b10110011);  
wait(rx_done);  
#20 ;  
t_bytes (8'b11001010);  
wait(tx_done);  
#40;  
$finish;  
end  
task t_bytes(input [7:0]data);  
begin  
wait(!tx_busy);  
tx_data = data;  
start=1;  
#30 start=0;  
end  
endtask  
  
endmodule
```